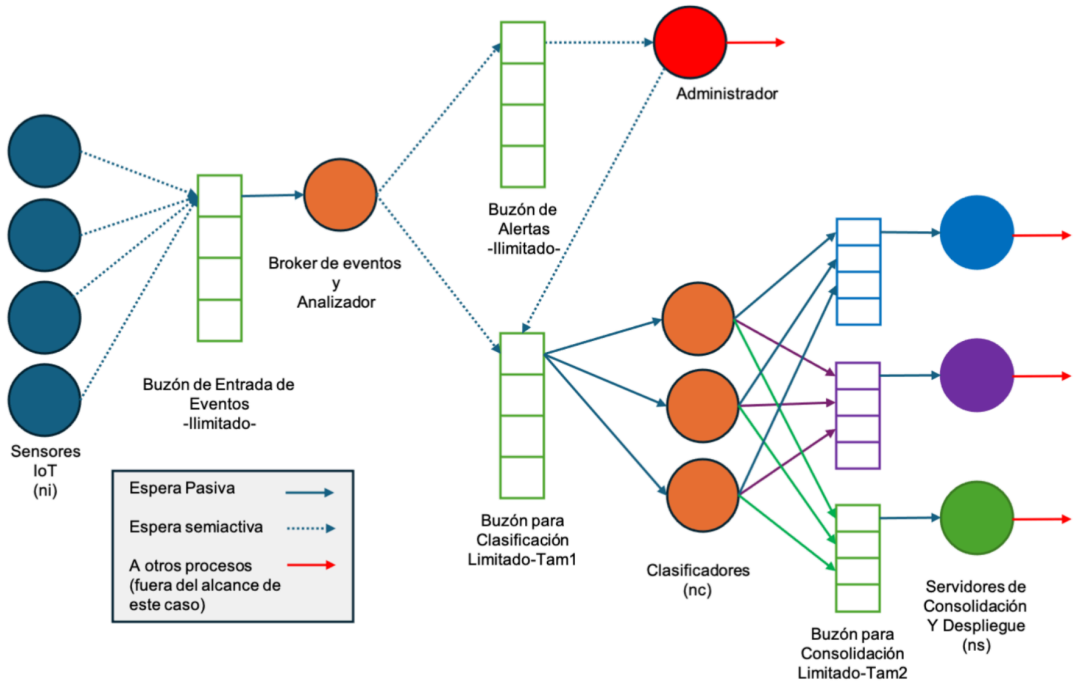
Caso 3 TIC

Tomás Luengas 202422456
Sara Poveda 202422288

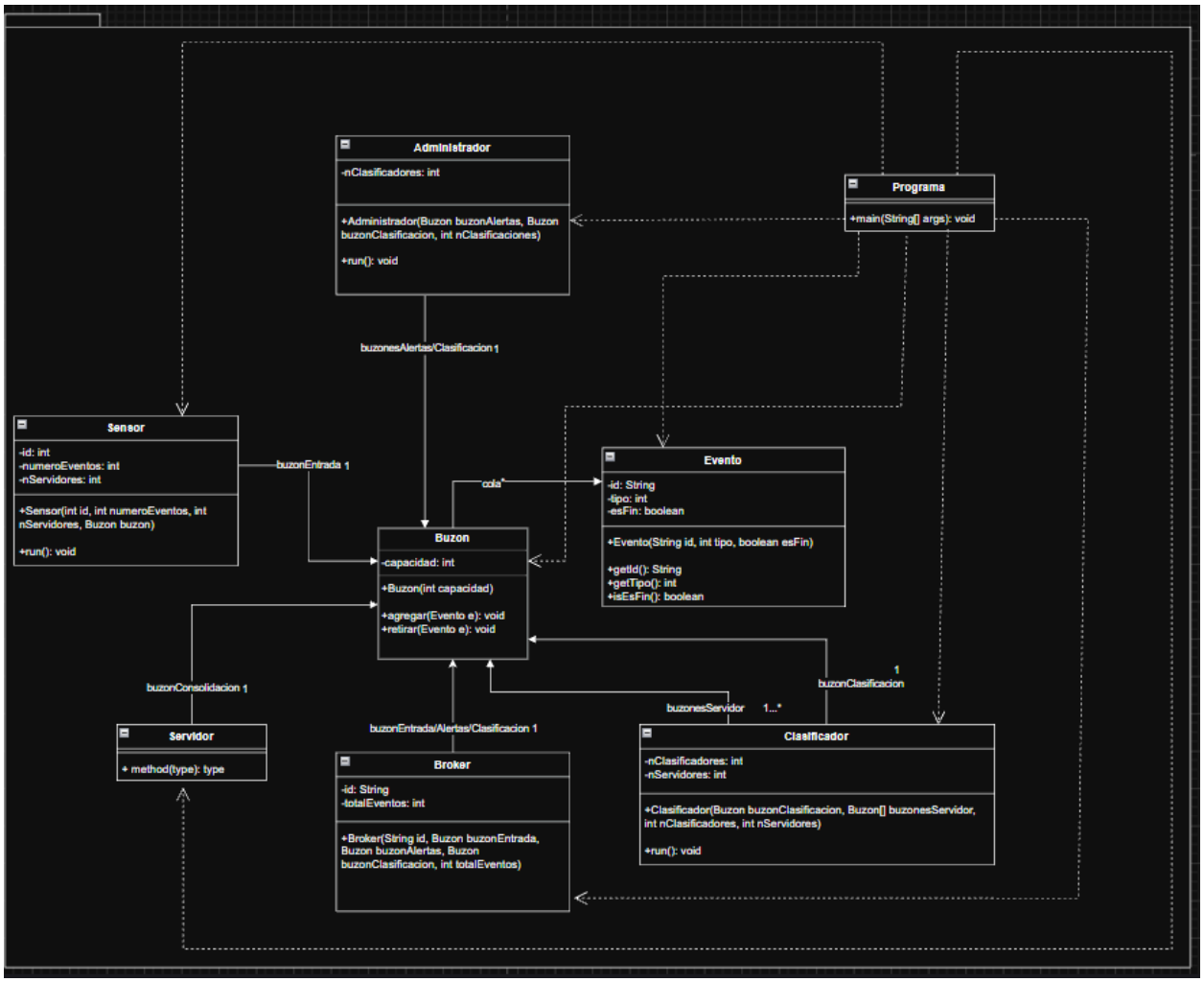


1. **Diseño y Funcionamiento del Sistema**
   a. *Diagrama de clases:*

El sistema está estructurado bajo un modelo de arquitectura multihilo, el cual consiste en la distribución de tareas entre diversas "máquinas" (hilos) que operan de forma independiente sin hablarse directamente entre sí.

En este caso específico, ninguna de las clases principales (como Sensores, Broker o Servidores) tiene contacto directo con las demás. En su lugar, utilizan la clase “Buzón”, que se comporta como una clase intermedia o puente. Este componente es el que permite que las máquinas se comuniquen y coordinen, entregando y recibiendo información de manera segura sin romper su independencia.

Caso 3 TIC
Tomás Luengas 202422456
Sara Poveda 202422288



Basados en el UML presentado se pueden identificar los siguientes elementos:

Los bloques de clases:

- Estas entidades cumplen cada una con una parte fundamental de la implementación, contando con sus atributos y metodos propios. Estas clases tienen los nombres de “Sensor”, “Broker”, “Administrador”, “Clasificador”, “Servidores”, “Buzon”, “Evento” y “Programa”.

Buzon como recurso compartido:

- Buzon en esta implementacion, como se puede ver en el diagrama de clases, se presenta como la posición central del programa que, como se dijo anteriormente, juega el rol de “banda trasportadora entre las otras clases”.

Representación del Flujo de Datos:

- La clase Evento aparece vinculada de forma transversal a los procesos de mensajería, representándose como el objeto que es transportado y procesado a través de las diferentes etapas del sistema.

  Clase de control:
- La clase “Programa” en la implementacion se muestra como el eje central en donde todos componentes implementados se conectan mediante lineas de control a los hilos de ejecucion.

b. *Funcionamiento del sistema:*

El funcionamiento del sistema se rige por un flujo de trabajo coordinado donde cada entidad aplica una lógica de transformación y filtrado sobre los datos:

Ciclo de Producción y Carga de Trabajo: El proceso inicia en los Sensores, los cuales no solo generan eventos, sino que determinan del mensaje asignándole un tipo específico antes de enviarlo al Buzón de Entrada.

Filtrado Del Broker: En esta parte de la implementacion, el Broker actúa como un primer filtro de calidad. El sistema no envía todo a clasificar; existe una lógica de desvío donde, mediante un cálculo aleatorio, se decide si un evento sigue el flujo normal o si se cataloga como una anomalía y se desvía al Buzón de Alertas.

Gestión de Excepciones y Normalización: El siguiente paso correspondiente al Administrador es la
capacidad de "reincorporar" eventos al flujo principal. Si un evento en el buzón de alertas supera un segundo filtro de validación, el Administrador lo envía al Buzón de Clasificación, funcionando como un mecanismo de recuperación de datos.

Sincronización por Estados de Espera: El funcionamiento real se basa en la espera pasiva. Cuando una clase intenta retirar un evento de un buzón vacío, el hilo entra en un estado de suspensión automática. No es sino hasta que otra clase agrega un evento que el sistema genera una señal de notificación que "despierta" a las máquinas encargadas de la siguiente etapa.

Mecanismo de Fin del programa: El sistema garantiza un apagado limpio mediante la técnica de "mensajes de control". Cuando el flujo de producción termina, se genera un evento especial de FIN. Este mensaje viaja por todas las etapas asegurando que cada máquina se apague solo después de haber procesado el último evento pendiente en su buzón.

Simulación de Procesamiento Final: En la última etapa, los Servidores ejecutan la carga de trabajo simulando tiempos de respuesta variables. Esto permite observar cómo los buzones intermedios gestionan la acumulación de mensajes cuando la velocidad de producción es mayor a la velocidad de procesamiento.

2. **Estrategia de concurrencia y Sincronización**
   a. *Sensores - Broker (Buzón Entrada):*
   Como este buzón tiene capacidad ilimitada, un Sensor jamás se va a encontrar con la restricción de "Buzón Lleno". Por lo tanto, nunca se bloquea al intentar depositar y hacer un yield() aquí sería totalmente redundante. Del otro lado, el Bróker sí usa espera pasiva wait(); si no hay eventos, simplemente se duerme liberando el procesador. Apenas un Sensor mete un evento, lanza un NotifyAll() y el Bróker arranca de nuevo.

   b. *Broker - Administrador (Buzón alertas):*
   La lógica es idéntica al caso anterior porque este buzón también es ilimitado. Poner al Bróker a hacer esperas semiactivas no tiene sentido porque sus depósitos siempre son instantáneos. Para optimizar recursos, el Administrador usa espera pasiva wait() y se queda dormido si no hay alertas sospechosas que revisar. Es el clásico modelo productor-consumidor.

   c. *Broker/Administrador - Clasificadores (Buzón clasificación):*
   ¡Aquí la cosa cambia! Como este buzón sí tiene un límite de capacidad (tam1), implementamos nuestra espera semiactiva. Creamos el método intentarAgregar() que verifica el espacio en una sola operación atómica. Si el buzón está lleno, el Bróker o el Administrador hacen un Thread.yield() para ceder la CPU y volver a intentarlo. Con esto evitamos que se queden trabados en un wait() y mantenemos el sistema altamente disponible incluso bajo estrés.

   d. *Clasificadores - Servidores:*
   Para esta última etapa volvimos a la espera pasiva estricta en ambos lados. Como este buzón también es limitado (tam2), si se llena, el Clasificador hace wait() para frenar y no causar un desbordamiento de memoria. Por otro lado, si el buzón está vacío, el Servidor hace wait() para no desperdiciar ciclos de CPU. Todo fluye usando notifyAll() para despertarse mutuamente de forma ordenada.

   e. *Justificación NotifyAll():*
   No fue por azar, lo hicimos para blindar el sistema contra deadlocks. Como todos los hilos comparten el monitor (synchronized), si usáramos notify() el sistema podría despertar a ciegas a un hilo del mismo bando (por ejemplo, un productor

despertando a otro productor). Si ese segundo productor ve que el buzón sigue lleno, se vuelve a dormir, y sin consumidores despiertos, ¡todo el sistema se congela para siempre!. Con notifyAll() aseguramos que todos escuchen el llamado y que el flujo nunca se detenga.

3. **Pruebas y validación**

Para evaluar el correcto funcionamiento del sistema, se diseñaron tres configuraciones base (archivos .txt) que simulan diferentes niveles de carga y estrés sobre la arquitectura. A continuación, se detallan los parámetros de cada archivo y, posteriormente, se documentan los resultados observados para cada escenario de prueba utilizando estos entornos.

Se crearon los siguientes 3 archivos:

- nombre_prueba.txt
  Valores:
  - Número sensores (ni)
  - Número base de eventos (base)
  - Número de clasificadores (nc)
  - Número de servidores (ns)
  - Capacidad Buzón Clasificación (Tam1)
  - Capacidad Buzón Consolidación (Tam 2)

- prueba_feliz.txt (Validación de flujo normal)
  Valores: 3,10, 2, 2, 50, 50
  Objetivo: Buzones gigantes (50) vs pocos eventos (30). Evita el estrés y nos permite validar la lógica matemática de desvíos sin ruido.

- prueba_cuello_botella.txt (Prueba de estrés y de bloqueos)
  Valores: 6, 20, 2, 2, 1, 1
  Objetivo: Buzones minúsculos (1) vs alta carga (120 eventos). Obliga a la arquitectura a frenar y probar la espera semiactiva y pasiva al límite. Así podemos verificar que no hayan *deadlocks* y que ambas esperas funcionen de la manera adecuada.

- prueba_terminación.txt (Validación de apagado)
  Valores: 2, 5, 10, 5, 20, 20
  Objetivo: Muchos servidores (15) vs poquísimos eventos (10). Evalúa si el mensaje de "FIN" es capaz de apagar el sistema sin dejar consumidores huérfanos.

a. *Escenarios de Integridad y Flujo de Datos (Todos con prueba_feliz.txt)*

i. <u>Contabilidad total de eventos</u>

Para validar que no existe pérdida de datos, implementamos contadores en el Administrador y los Servidores usando un contador. La prueba es un éxito si el total generado es igual a la suma de descartados y procesados. Como se evidencia en consola, el sistema contabiliza el 100 % del flujo sin perder información.

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

zukach@Saras-MacBook src % java caso3.Programa prueba_feliz.txt
Servidor procesó evento: 3-evento25
Servidor procesó evento: 3-evento28
Servidor procesó evento: 3-evento27
Servidor procesó evento: 2-evento19
Servidor procesó evento: 3-evento10
Servidor recibió señal de fin.
Servidor procesó evento: 3-evento29
Servidor recibió señal de fin.

=== CONTABILIDAD TOTAL DE EVENTOS ===
Total Esperado (Generados por Sensores): 60
Total Procesados (Servidores): 52
Total Descartados (Administrador): 8
Suma Real (Procesados + Descartados): 60
ESTADO: EXITO - No se perdió ningún evento.
zukach@Saras-MacBook src %
                                                  Tomás Luengas
```

ii. Filtro del broker y Filtro de Administrador

Como podemos ver en el siguiente código:

```
int numero = (int)(Math.random() * 201);
// es anómalo → buzonAlertas
if (numero % 8 == 0) {
    buzonAlertas.agregar(evento);
    System.out.println("Broker " + id + " envió alerta: " + even
```

Cuando un evento es múltiplo de 8, debería ser reconocido como anómalo. Si vemos en nuestra ejecución, sí hay ocasiones en las que esto sucede, y el Broker envía al buzón correspondiente:

```
zukach@Saras-MacBook src % java caso3.Prog
Broker Broker1 procesó evento: 3-evento21
176
Broker Broker1 envió alerta: 3-evento21. Para el numero: 176
Broker Broker1 procesó evento: 3-evento22
Broker Broker1 envió a clasificar: 3-evento22
Administrador: 3-evento21 era malicioso. Fue descartado: 3-evento21. Para el numero generado: 13
Broker Broker1 procesó evento: 3-evento23
Broker Broker1 envió a clasificar: 3-evento23
Broker Broker1 procesó evento: 3-evento24
Broker Broker1 envió a clasificar: 3-evento24
Broker Broker1 procesó evento: 3-evento25
Broker Broker1 envió a clasificar: 3-evento25
Broker Broker1 procesó evento: 3-evento26
Broker Broker1 envió a clasificar: 3-evento26
Broker Broker1 procesó evento: 3-evento27
Broker Broker1 envió a clasificar: 3-evento27
Broker Broker1 procesó evento: 3-evento28
104
Broker Broker1 envió alerta: 3-evento28. Para el numero: 104
Broker Broker1 procesó evento: 3-evento29
Broker Broker1 envió a clasificar: 3-evento29
Broker Broker1 procesó evento: 1-evento0
Broker Broker1 envió a clasificar: 1-evento0
Broker Broker1 procesó evento: 1-evento1
Broker Broker1 envió a clasificar: 1-evento1
Broker Broker1 procesó evento: 1-evento2
Broker Broker1 envió a clasificar: 1-evento2
Broker Broker1 procesó evento: 1-evento3
Broker Broker1 envió a clasificar: 1-evento3
Broker Broker1 procesó evento: 1-evento4
24
Broker Broker1 envió alerta: 1-evento4. Para el numero: 24
Broker Broker1 procesó evento: 1-evento5
```

Por otra parte, si vemos el código de Administrador ocurre:

```
int numero = (int)(Math.random() * 21);
// es anómalo → Descartar
if (numero % 4 != 0) {
    System.out.println("Administrador: " + evento.getId() + " era malicioso.
```

Cuando el número generado es diferente a un múltiplo de 4, como dice el enunciado, se debe descartar (no hacer nada con ello). Esto ocurre:

```
Broker Broker1 envió a clasificar: 2-evento19
Administrador: 3-evento28 era inofensivo. Enviado a buzonClasificacion.
Administrador: 1-evento4 era malicioso. Fue descartado: 1-evento4. Para el numero generado: 2
Administrador: 1-evento8 era malicioso. Fue descartado: 1-evento8. Para el numero generado: 9
Administrador: 1-evento9 era inofensivo. Enviado a buzonClasificacion.
Administrador: 2-evento15 era malicioso. Fue descartado: 2-evento15. Para el numero generado: 1
Administrador recibió señal de fin.
```

Por lo que podemos determinar que estos filtros están funcionando de la manera indicada.

iii. Enrutamiento correcto

La validación del enrutamiento de la arquitectura se comprueba transversalmente a partir de los resultados de las pruebas de Contabilidad Total y Filtros de Seguridad. Al evidenciar que el 100% de los eventos generados alcanzan un estado final válido (procesados por los Servidores o descartados por el Administrador) y que los desvíos condicionales operan estrictamente según las reglas de negocio, se certifica que la comunicación a través de los buzones enruta la información de manera íntegra, sin fugas ni interbloqueos.

b. *Escenarios de sincronización y concurrencia (Todos con prueba_cuello_botella.txt)*

i. Bloqueo por buzones llenos (Productor rápido, consumidor lento)

Forzamos un cuello de botella en el Buzón de Consolidación volviendo súper lentos a los Servidores (Thread.sleep()). Al llenarse la capacidad (tam2 = 1), los Clasificadores no gastan CPU a lo loco, sino que entran a un estado de espera pasiva con wait(). Ahí se quedan bloqueados hasta que el Servidor saca un evento y hace el notifyAll().

```
zukach@Saras-MacBook src % java caso3.Programa prueba_cuello_botella.txt
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
Servidor procesó evento: 3-evento15
Evento ya hace parte del buzon de servidores.
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
Servidor procesó evento: 6-evento0
Evento ya hace parte del buzon de servidores.
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
Servidor procesó evento: 1-evento6
Evento ya hace parte del buzon de servidores.
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
Servidor procesó evento: 1-evento7
Evento ya hace parte del buzon de servidores.
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
Servidor procesó evento: 6-evento1
Evento ya hace parte del buzon de servidores.
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
PRODUCTOR BLOQUEADO: Buzón lleno. Esperando a que el Servidor lento libere espacio...
Servidor procesó evento: 3-evento16
```

ii. Espera por buzones vacíos (Productor lento, consumidor rápido)

Aquí hicimos lo contrario: volvemos lentos a los Sensores. El Bróker vacía el Buzón de Entrada instantáneamente. Al encontrarlo vacío, el Bróker usa wait() para entrar en espera pasiva (mostrando "Consumidor Bloqueado"). Demostramos que el diseño protege los recursos del sistema cuando hay muy poco flujo de trabajo.

```
zukach@Saras-MacBook src % java caso3.Programa prueba_cuello_botella.txt
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
Sensor 2 generó evento: 2-evento0
Sensor 4 generó evento: 4-evento0
Sensor 1 generó evento: 1-evento0
Sensor 6 generó evento: 6-evento0
Sensor 3 generó evento: 3-evento0
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
Sensor 5 generó evento: 5-evento0
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
CONSUMIDOR BLOQUEADO: Buzón vacío. Bróker esperando a que el Sensor lento genere algo...
```

iii. <u>Condición de carrera</u>

Como vimos en la prueba de Contabilidad Total, confirmamos que en nuestra arquitectura no ocurren condiciones de carrera. Esto se debe a que usamos synchronized en las partes críticas del código (como los métodos de los buzones) para asegurar que solo un hilo acceda a la vez. Si no hubiéramos puesto este "candado", los hilos se habrían sobreescrito entre sí. Al ver que nuestra suma final dio exacta, demostramos empíricamente que el sistema no sufre de estas colisiones.

c. *Escenarios de Terminación (Todos con prueba_terminacion.txt)*

i. <u>Propagación del evento “FIN”</u>

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

zukach@Saras-MacBook src % java caso3.Programa prueba_termin
Broker Broker1 envió a clasificar: 1-evento3
Broker Broker1 procesó evento: 2-evento6
Evento ya hace parte del buzon de servidores.
Evento ya hace parte del buzon de servidores.
Broker Broker1 envió a clasificar: 2-evento6
Broker Broker1 procesó evento: 1-evento4
Broker Broker1 envió a clasificar: 1-evento4
Broker Broker1 procesó evento: 2-evento7
Broker Broker1 envió a clasificar: 2-evento7
Broker Broker1 procesó evento: 2-evento8
Evento ya hace parte del buzon de servidores.
Evento ya hace parte del buzon de servidores.
Evento ya hace parte del buzon de servidores.
Broker Broker1 envió a clasificar: 2-evento8
Evento ya hace parte del buzon de servidores.
Broker Broker1 procesó evento: 2-evento9
Broker Broker1 envió a clasificar: 2-evento9
Evento ya hace parte del buzon de servidores.
Administrador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Clasificador recibió señal de fin.
Servidor procesó evento: 2-evento5
Servidor procesó evento: 1-evento0
Servidor procesó evento: 1-evento2
Servidor procesó evento: 2-evento6
Servidor procesó evento: 1-evento1
Servidor procesó evento: 2-evento8
Servidor procesó evento: 1-evento4
Servidor procesó evento: 2-evento0
Servidor procesó evento: 2-evento4
Servidor recibió señal de fin.
Servidor procesó evento: 2-evento2
Servidor procesó evento: 1-evento3
Servidor recibió señal de fin.
```

Los 10 Clasificadores despiertos recibieron todos la señal final después de que el administrador la recibiera. La cascada de “FIN”s se mantiene. Además, en esta ejecución en específico vimos como solo 2 servidores reciben consultas por que esto es determinado por un número generado en el sensor, por lo que son ellos quienes terminan, y dan finalización al sistema.

ii. <u>Buzones vacíos al final</u>

Para cerrar nuestro protocolo de apagado, no nos bastaba con que los hilos simplemente dejaran de imprimir; teníamos que asegurarnos de que no dejaran un desastre en la memoria. Si el sistema se apagara de golpe apenas ve el primer mensaje de "FIN", los buzones se quedarían llenos de eventos a medio procesar, perdiendo datos valiosos justo en el último segundo.

En nuestra ejecución final pudimos comprobar que, al terminar el programa, el tamaño de las colas en absolutamente todos los buzones (Entrada, Alertas, Clasificación y Consolidación) queda exactamente en cero. Esto demuestra que nuestra propagación del "FIN" funciona perfecto: obliga a cada máquina a trabajar hasta exprimir el último evento que tenga pendiente en su buzón antes de darle permiso para apagarse y cerrar el ciclo. Así garantizamos un cierre completamente limpio y sin dejar hilos colgados.